

---

---

# Session 08 (AIML) - Assignment Questions

Name: **Vaishnavi Praveen Rathi**

College: **Zeal Polytechnic, Narhe, Pune**

Branch: **Artificial Intelligence and Machine Learning (AIML)**

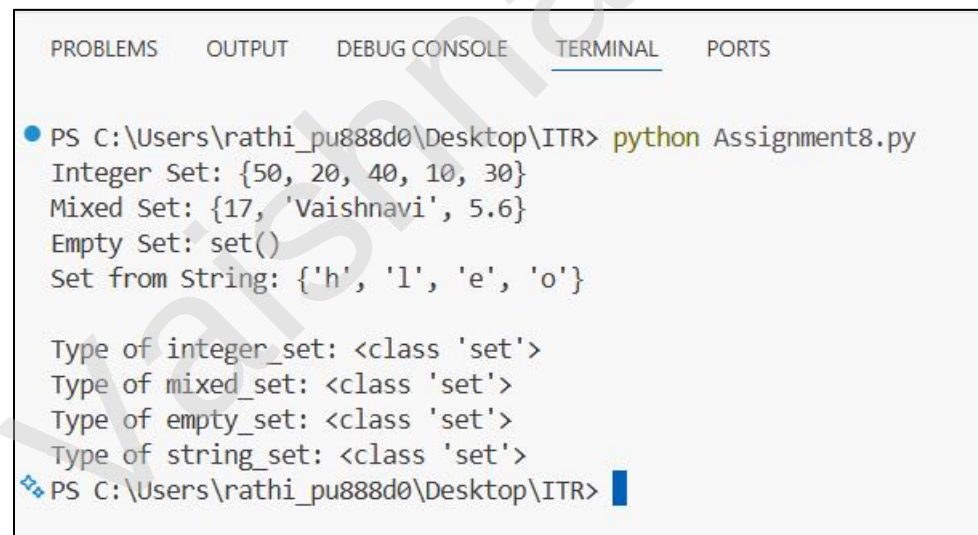
Domain: **AIML (Artificial Intelligence and Machine Learning)**

---

Q1. Creating Sets

---

**Output Screenshot:**



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment8.py
Integer Set: {50, 20, 40, 10, 30}
Mixed Set: {17, 'Vaishnavi', 5.6}
Empty Set: set()
Set from String: {'h', 'l', 'e', 'o'}

Type of integer_set: <class 'set'>
Type of mixed_set: <class 'set'>
Type of empty_set: <class 'set'>
Type of string_set: <class 'set'>
PS C:\Users\rathi_pu888d0\Desktop\ITR> |
```

---

## Program Code:

```
""" ASSIGNMENT 8 """
```

```
'''
```

Q1. (Creating Sets)

Write a program to create the following sets:

- a) A set with 5 integers.
- b) A set with mixed data types (integer, string, float).
- c) An empty set using the correct method.
- d) A set from the string "hello" using the set() constructor.

Print each set and its type.

Add comments explaining why sets automatically remove duplicates.

```
'''
```

```
# a) Creating a set with 5 integers
```

```
integer_set = {10, 20, 30, 40, 50}
```

```
# b) Creating a set with mixed data types
```

```
mixed_set = {17, "Vaishnavi", 5.6}
```

```
# c) Creating an empty set using set()
```

```
empty_set = set()
```

```
# d) Creating a set from the string "hello"
```

```
string_set = set("hello")
```

```
# Print all sets
```

```
print("Integer Set:", integer_set)
```

```
print("Mixed Set:", mixed_set)
```

```
print("Empty Set:", empty_set)
```

```
print("Set from String:", string_set)
```

```
print()
```

```
# Print types
```

```
print("Type of integer_set:", type(integer_set))
```

```
print("Type of mixed_set:", type(mixed_set))
```

```
print("Type of empty_set:", type(empty_set))
```

```
print("Type of string_set:", type(string_set))
```

### "Explanation in Comments"

```
# Sets are created using curly braces {}.  
# An empty set must be created using set().  
# Using {} creates an empty dictionary, not a set.  
  
# set("hello") creates a set of unique characters from the string.  
  
# Sets automatically remove duplicate values.  
# Example: set("hello") contains only one 'l' even though it appears twice.  
# This is because sets store only unique elements.  
  
# Sets are unordered, so the output order may be different each time.
```

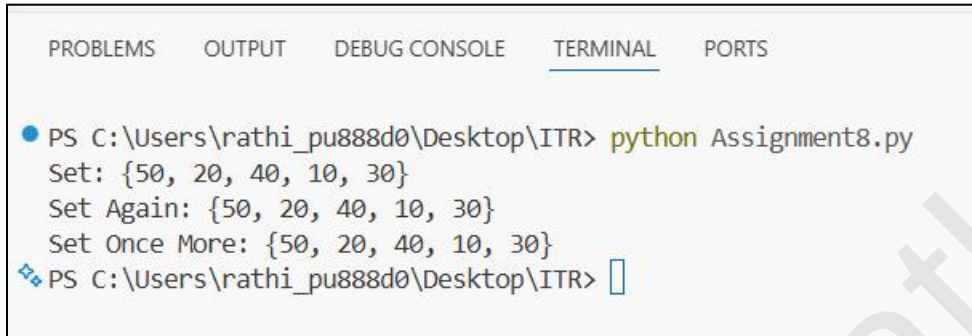
---

---

## Q2. Characteristics of Sets

---

### Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment8.py
Set: {50, 20, 40, 10, 30}
Set Again: {50, 20, 40, 10, 30}
Set Once More: {50, 20, 40, 10, 30}
PS C:\Users\rathi_pu888d0\Desktop\ITR> 
```

### Program Code:

""" ASSIGNMENT 8 """

'''

Q2. (Characteristics of Sets)

Create a set:

numbers = {10, 20, 30, 20, 40, 10, 50}

Print the set and observe the output.

Demonstrate that sets are unordered by printing the set multiple times.

Try to access the first element using indexing (numbers[0]) and explain the error in comments.

'''

# Creating a set with duplicate values

numbers = {10, 20, 30, 20, 40, 10, 50}

# Print the set

print("Set:", numbers)

# Print the set multiple times

print("Set Again:", numbers)

print("Set Once More:", numbers)

# The following statement causes an error

# print(numbers[0])

# Error:  
# TypeError: 'set' object is not subscriptable

"""Explanation in Comments"""

# Sets automatically remove duplicate values.  
# In this set, 10 and 20 appear more than once,  
# but only one copy of each is stored.

# Sets are unordered collections.  
# Elements are not stored in a fixed position.

# Because sets are unordered, indexing is not supported.  
# Therefore, numbers[0] causes a TypeError.

# To access elements of a set, we typically use loops  
# or convert the set to a list if indexing is required.

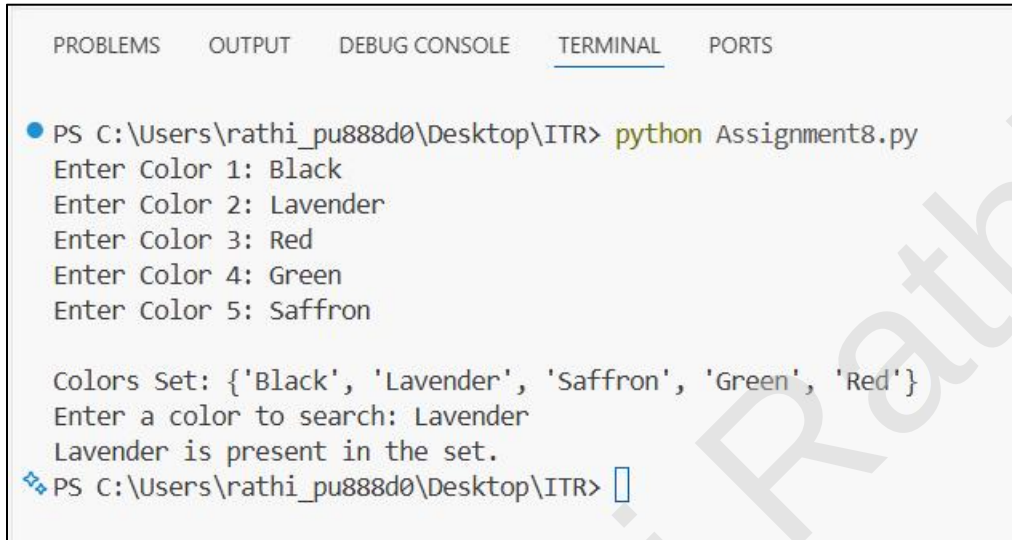
---

---

### Q3. Membership Testing

---

#### Output Screenshot:



The screenshot shows a terminal window with tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The TERMINAL tab is active. The prompt is PS C:\Users\rathi\_pu888d0\Desktop\ITR>. The user runs python Assignment8.py. The program prompts for 5 colors: Enter Color 1: Black, Enter Color 2: Lavender, Enter Color 3: Red, Enter Color 4: Green, Enter Color 5: Saffron. It then displays the set: Colors Set: {'Black', 'Lavender', 'Saffron', 'Green', 'Red'}. The user is prompted to enter a color to search: Enter a color to search: Lavender. The program outputs: Lavender is present in the set. The prompt returns to PS C:\Users\rathi\_pu888d0\Desktop\ITR>.

---

#### Program Code:

```
""" ASSIGNMENT 8 """
```

```
'''
```

```
Q3. (Membership Testing)
```

```
Write a program that:
```

```
Takes a set of 5 colors as input from the user.
```

```
Asks the user to enter a color name.
```

```
Checks whether the color is present in the set using in and not in.
```

```
Prints appropriate messages.
```

```
'''
```

```
# Create an empty set
```

```
colors = set()
```

```
# Take 5 colors as input from the user
```

```
for i in range(5):
```

```
    color = input(f"Enter Color {i + 1}: ")
```

```
    colors.add(color)
```

```
# Display the set
```

```
print("\nColors Set:", colors)
```

```
# Take a color to search
search_color = input("Enter a color to search: ")
```

```
# Check membership using in and not in
if search_color in colors:
    print(search_color, "is present in the set.")
elif search_color not in colors:
    print(search_color, "is not present in the set.")
```

"Explanation in Comments"

```
# add() is used to add elements to a set.
# The 'in' operator checks whether an element exists in the set.
# The 'not in' operator checks whether an element does not exist in the set.
# Sets automatically remove duplicate values entered by the user.
# Membership testing in sets is fast and efficient.
.
```

---

---

Q4. add(), remove(), discard()

---

### Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment8.py
Original Set: {'banana', 'mango', 'apple'}
After add('orange'): {'banana', 'mango', 'orange', 'apple'}
After add('banana') Again: {'banana', 'mango', 'orange', 'apple'}
After remove('mango'): {'banana', 'orange', 'apple'}
After discard('grape'): {'banana', 'orange', 'apple'}
PS C:\Users\rathi_pu888d0\Desktop\ITR>
```

### Program Code:

```
""" ASSIGNMENT 7 """
```

```
'''
```

Q4. (Slicing Tuples)

Given the tuple:

```
numbers = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
```

Using slicing, print:

- a) Elements from index 2 to 7
- b) First 5 elements
- c) Last 4 elements
- d) Every second element
- e) The entire tuple in reverse order

Add comments explaining the slicing syntax.

```
'''
```

```
# Creating the tuple
```

```
numbers = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
```

```
# a) Elements from index 2 to 7
```

```
print("Elements from Index 2 to 7:", numbers[2:8])
```

```
# b) First 5 elements
```

```
print("First 5 Elements:", numbers[:5])
```



```
# c) Last 4 elements
print("Last 4 Elements:", numbers[-4:])

# d) Every second element
print("Every Second Element:", numbers[::2])

# e) Entire tuple in reverse order
print("Tuple in Reverse Order:", numbers[::-1])

"Explanation in Comments"

# Tuple slicing uses the syntax [start:end:step].

# start -> Starting index (inclusive).
# end -> Ending index (exclusive).
# step -> Number of positions to move.

# numbers[2:8] returns elements from index 2 to 7.
# numbers[:5] returns the first 5 elements.
# numbers[-4:] returns the last 4 elements.
# numbers[::2] returns every second element.
# numbers[::-1] reverses the tuple.

# Slicing allows us to access a portion of a tuple easily.
```

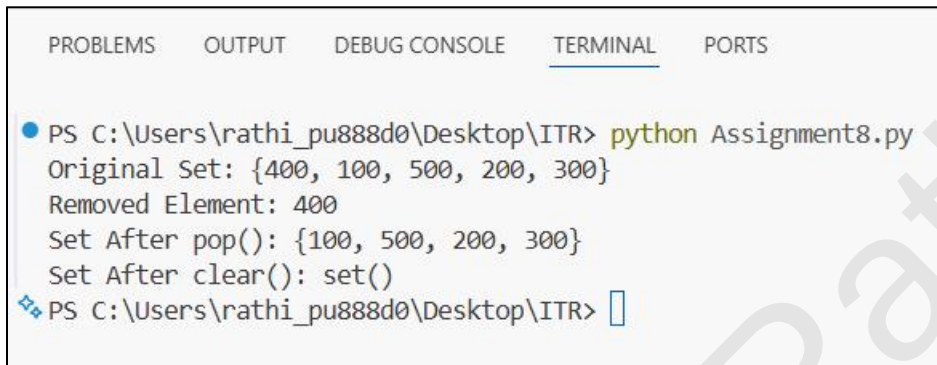
---

---

Q5. pop() and clear()

---

### Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment8.py
Original Set: {400, 100, 500, 200, 300}
Removed Element: 400
Set After pop(): {100, 500, 200, 300}
Set After clear(): set()
PS C:\Users\rathi_pu888d0\Desktop\ITR>
```

### Program Code:

```
""" ASSIGNMENT 8 """
```

```
'''
```

```
Q5. (pop() and clear())
```

```
Create a set:
```

```
s = {100, 200, 300, 400, 500}
```

```
Use pop() to remove and print one element.
```

```
Print the set after popping.
```

```
Clear the entire set using clear().
```

```
Print the final empty set.
```

```
Add comments explaining the difference between remove(), discard(), and pop().
```

```
'''
```

```
# Creating the set
```

```
s = {100, 200, 300, 400, 500}
```

```
print("Original Set:", s)
```

```
# Remove and print one element using pop()
```

```
removed_element = s.pop()
```

```
print("Removed Element:", removed_element)
```

```
# Print the set after pop()
```

```
print("Set After pop():", s)
```

```
# Clear the entire set  
s.clear()
```

```
# Print the empty set  
print("Set After clear():", s)
```

```
"Explanation in Comments"
```

```
# pop() removes and returns a random element from the set.  
# Since sets are unordered, we cannot predict which element will be removed.
```

```
# clear() removes all elements from the set and makes it empty.
```

```
# Difference between remove(), discard(), and pop():
```

```
# remove(value)  
# Removes the specified value.  
# Raises a KeyError if the value is not present.
```

```
# discard(value)  
# Removes the specified value.  
# Does not raise an error if the value is not present.
```

```
# pop()  
# Removes and returns a random element.  
# Does not require a value as an argument.
```

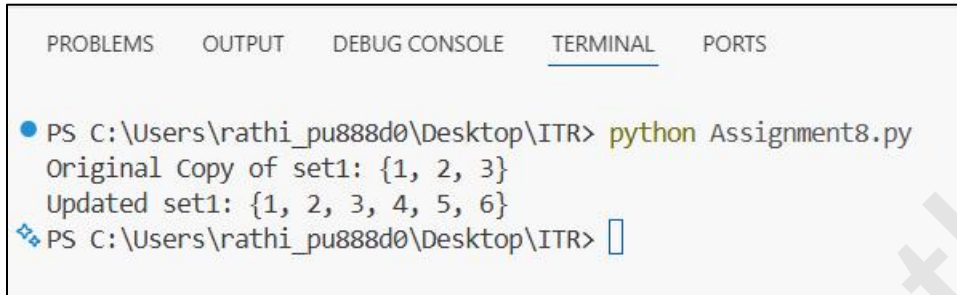
---

---

Q6. update() and copy()

---

**Output Screenshot:**



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment8.py
Original Copy of set1: {1, 2, 3}
Updated set1: {1, 2, 3, 4, 5, 6}
PS C:\Users\rathi_pu888d0\Desktop\ITR>
```

**Program Code:**

```
""" ASSIGNMENT 8 """
```

```
'''
```

Q6. (update() and copy())

Create two sets:

```
set1 = {1, 2, 3}
set2 = {3, 4, 5, 6}
```

Make a copy of set1.

Update set1 with elements from set2 using update().

Print both the original copy and the updated set.

```
'''
```

```
# Creating the sets
```

```
set1 = {1, 2, 3}
```

```
set2 = {3, 4, 5, 6}
```

```
# Make a copy of set1
```

```
set1_copy = set1.copy()
```

```
# Update set1 with elements from set2
```

```
set1.update(set2)
```

```
# Print the original copy and updated set
```

```
print("Original Copy of set1:", set1_copy)
```

```
print("Updated set1:", set1)
```

### "Explanation in Comments"

```
# copy() creates a duplicate of the set.  
# Changes made to the original set do not affect the copied set.  
  
# update() adds all elements from another set.  
# Duplicate elements are not added because sets store only unique values.  
  
# In this example, 3 is already present in set1,  
# so it appears only once in the updated set.
```

---

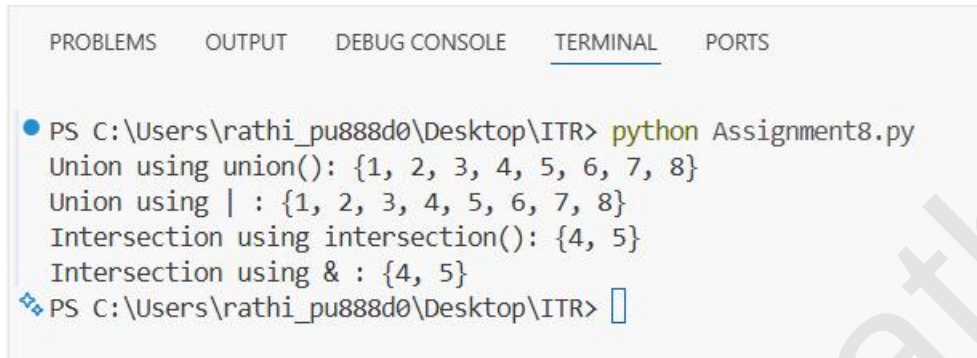
Vaishnavi Rathni

---

## Q7. Union and Intersection

---

### Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment8.py
Union using union(): {1, 2, 3, 4, 5, 6, 7, 8}
Union using | : {1, 2, 3, 4, 5, 6, 7, 8}
Intersection using intersection(): {4, 5}
Intersection using & : {4, 5}
PS C:\Users\rathi_pu888d0\Desktop\ITR>
```

### Program Code:

```
""" ASSIGNMENT 8 """
```

```
'''
```

```
Q7. (Union and Intersection)
```

```
Create two sets:
```

```
A = {1, 2, 3, 4, 5}
```

```
B = {4, 5, 6, 7, 8}
```

```
Print the union of A and B (using both .union() and | operator).
```

```
Print the intersection of A and B (using both .intersection() and & operator).
```

```
Add comments explaining what each operation does.
```

```
'''
```

```
# Creating the sets
```

```
A = {1, 2, 3, 4, 5}
```

```
B = {4, 5, 6, 7, 8}
```

```
# Union using union() method
```

```
print("Union using union():", A.union(B))
```

```
# Union using | operator
```

```
print("Union using | :", A | B)
```

```
# Intersection using intersection() method
```

```
print("Intersection using intersection():", A.intersection(B))
```

```
# Intersection using & operator  
print("Intersection using & :", A & B)
```

"Explanation in Comments"

```
# Union combines all unique elements from both sets.
```

```
# Duplicate elements are included only once.
```

```
# A.union(B) and A | B give the same result.
```

```
# Intersection returns only the common elements
```

```
# that are present in both sets.
```

```
# A.intersection(B) and A & B give the same result.
```

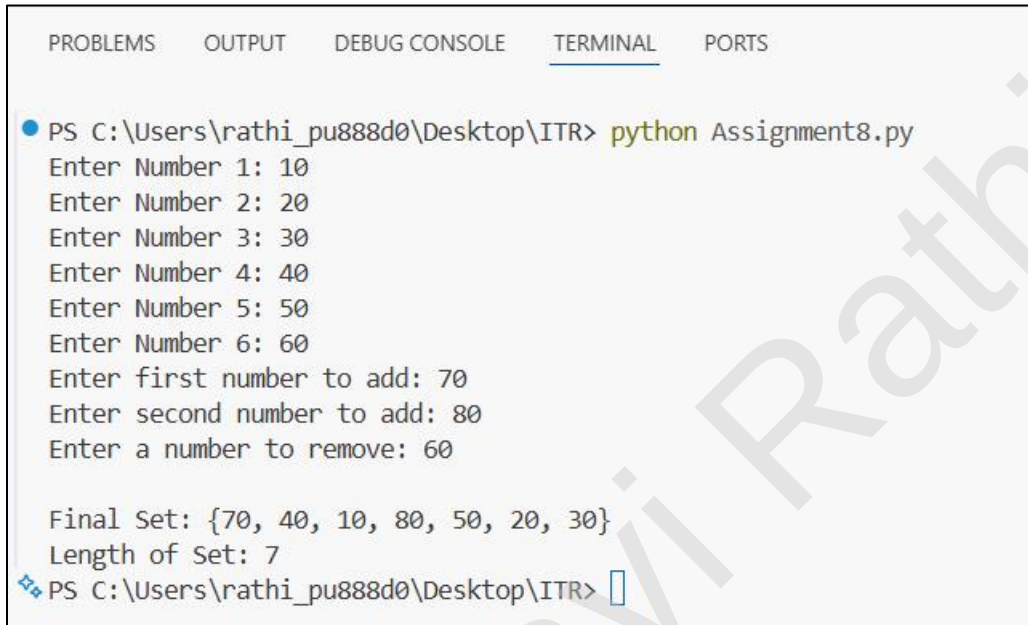
---

---

## Q8. Combined Methods

---

### Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment8.py
Enter Number 1: 10
Enter Number 2: 20
Enter Number 3: 30
Enter Number 4: 40
Enter Number 5: 50
Enter Number 6: 60
Enter first number to add: 70
Enter second number to add: 80
Enter a number to remove: 60

Final Set: {70, 40, 10, 80, 50, 20, 30}
Length of Set: 7
PS C:\Users\rathi_pu888d0\Desktop\ITR> 
```

### Program Code:

```
""" ASSIGNMENT 8 """
```

```
'''
```

```
Q8. (Combined Methods)
```

```
Write a program that:
```

```
Takes 6 numbers as input from the user and stores them in a set
(duplicates should be removed automatically).
```

```
Adds two more numbers using add().
```

```
Removes one number safely using discard().
```

```
Prints the final set and its length using len().
```

```
'''
```

```
# Create an empty set
numbers_set = set()
```



```

# Take 6 numbers as input from the user
for i in range(6):
    num = int(input(f"Enter Number {i + 1}: "))
    numbers_set.add(num)

# Add two more numbers
num1 = int(input("Enter first number to add: "))
num2 = int(input("Enter second number to add: "))

numbers_set.add(num1)
numbers_set.add(num2)

# Remove a number safely using discard()
remove_num = int(input("Enter a number to remove: "))
numbers_set.discard(remove_num)

# Print the final set
print("\nFinal Set:", numbers_set)

# Print the length of the set
print("Length of Set:", len(numbers_set))

"""Explanation in Comments"""

# add() is used to add an element to a set.

# Sets automatically remove duplicate values.
# If the same number is entered multiple times,
# it will appear only once in the set.

# discard() removes an element if it exists.
# No error occurs if the element is not present.

# len() returns the total number of unique elements in the set.

```

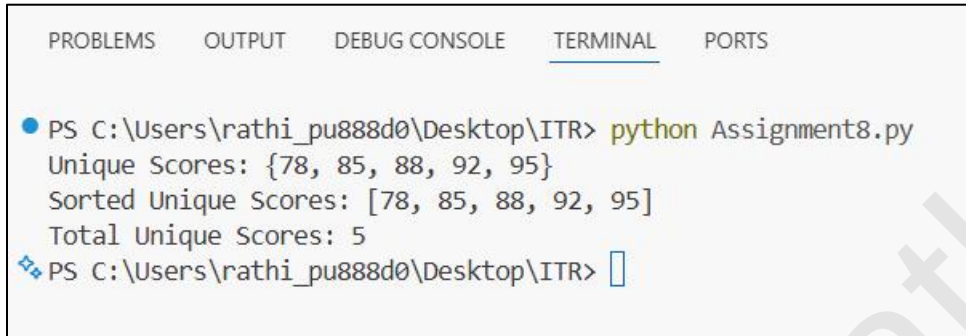
---

---

## Q9. Practical Application

---

### Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment8.py
Unique Scores: {78, 85, 88, 92, 95}
Sorted Unique Scores: [78, 85, 88, 92, 95]
Total Unique Scores: 5
PS C:\Users\rathi_pu888d0\Desktop\ITR> 
```

### Program Code:

```
""" ASSIGNMENT 8 """
```

```
'''
```

Q9. (Practical Application)

Write a program to remove duplicate marks from a list of student scores using a set:

```
scores = [85, 92, 78, 92, 85, 88, 95, 78]
```

Convert the list to a set.

Print the unique scores.

Convert the set back to a sorted list and print it.

Print the total number of unique scores.

```
'''
```

```
# Create the list of scores
```

```
scores = [85, 92, 78, 92, 85, 88, 95, 78]
```

```
# Convert the list to a set
```

```
unique_scores = set(scores)
```

```
# Print the unique scores
```

```
print("Unique Scores:", unique_scores)
```

```
# Convert the set back to a sorted list
```

```
sorted_scores = sorted(list(unique_scores))
```

```
# Print the sorted list
```

```
print("Sorted Unique Scores:", sorted_scores)
```

```
# Print the total number of unique scores
print("Total Unique Scores:", len(unique_scores))
```

'''Explanation in Comments'''

```
# set() removes duplicate values from the list automatically.
```

```
# unique_scores contains only unique marks.
```

```
# sorted() arranges the elements in ascending order
# and returns a new sorted list.
```

```
# len() returns the total number of unique scores.
```

```
# This technique is useful when duplicate data
# needs to be removed quickly.
```

```
# 3. Convert the list back into a tuple using tuple().
```

---

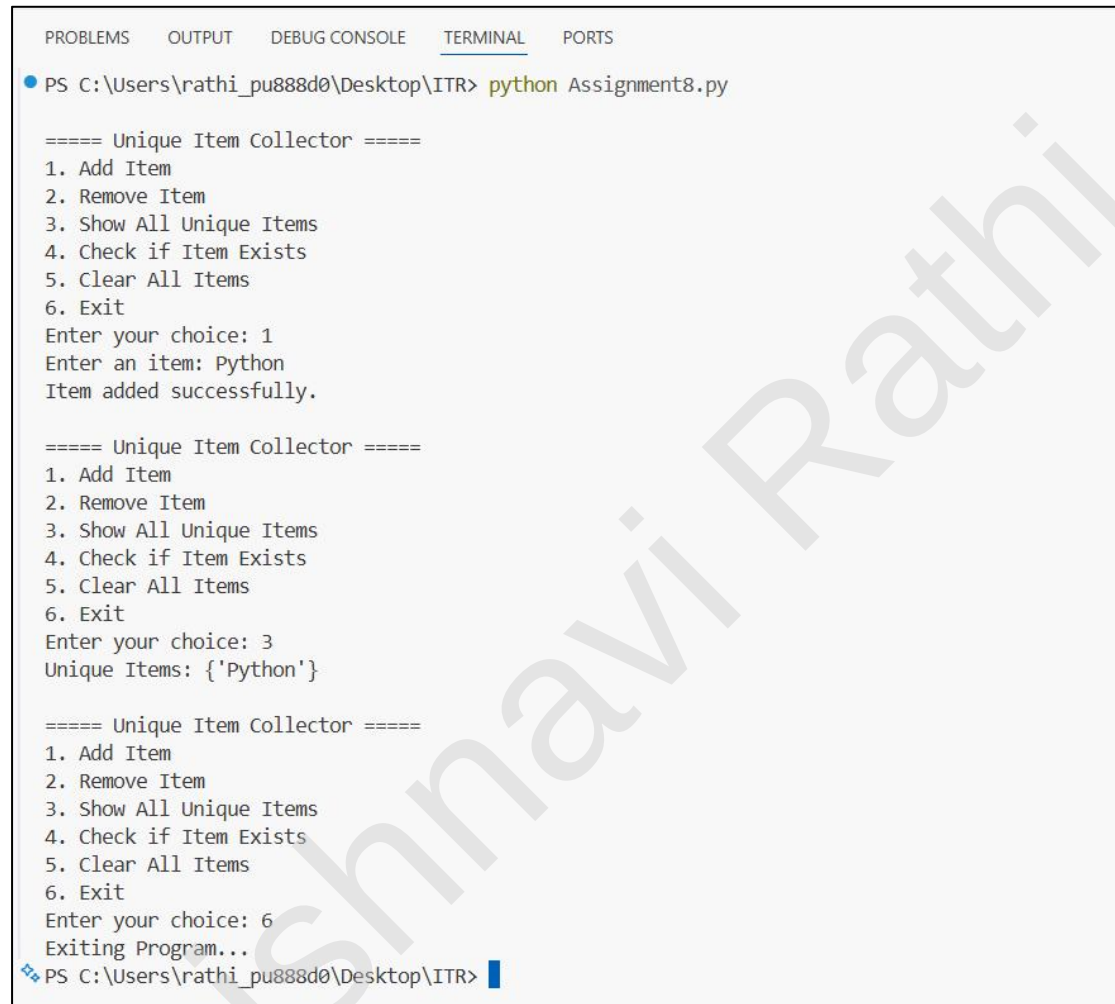
--

---

## Q10. Mini Project - Combined Concepts

---

### Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment8.py

===== Unique Item Collector =====
1. Add Item
2. Remove Item
3. Show All Unique Items
4. Check if Item Exists
5. Clear All Items
6. Exit
Enter your choice: 1
Enter an item: Python
Item added successfully.

===== Unique Item Collector =====
1. Add Item
2. Remove Item
3. Show All Unique Items
4. Check if Item Exists
5. Clear All Items
6. Exit
Enter your choice: 3
Unique Items: {'Python'}

===== Unique Item Collector =====
1. Add Item
2. Remove Item
3. Show All Unique Items
4. Check if Item Exists
5. Clear All Items
6. Exit
Enter your choice: 6
Exiting Program...
PS C:\Users\rathi_pu888d0\Desktop\ITR>
```

---

### Program Code:

```
""" ASSIGNMENT 8 """
```

```
'''
```

Q10. (Mini Project - Combined Concepts)

Create a Unique Item Collector program using sets:

Repeatedly ask the user to enter items.

Store all entered items in a set (duplicates should be ignored).

Menu:

1. Add item
2. Remove item
3. Show all unique items
4. Check if an item exists
5. Clear all items
6. Exit

Use a while loop for the menu and appropriate set methods.

'''

# Create an empty set to store unique items

items = set()

# Run the menu until the user chooses Exit

while True:

```
print("\n===== Unique Item Collector =====")
print("1. Add Item")
print("2. Remove Item")
print("3. Show All Unique Items")
print("4. Check if Item Exists")
print("5. Clear All Items")
print("6. Exit")
```

choice = int(input("Enter your choice: "))

# Add a new item

if choice == 1:

```
    item = input("Enter an item: ")
    items.add(item)
    print("Item added successfully.")
```

# Remove an item using discard()

elif choice == 2:

```
    item = input("Enter item to remove: ")
    items.discard(item)
    print("Item removed (if it existed).")
```

# Display all unique items

elif choice == 3:

```
    print("Unique Items:", items)
```

# Check whether an item exists

elif choice == 4:

```
    item = input("Enter item to search: ")
```

```
    if item in items:
```

```
        print(item, "exists in the set.")
```

```
    else:
```

```
print(item, "does not exist in the set.")

# Clear all items
elif choice == 5:
    items.clear()
    print("All items have been cleared.")

# Exit the program
elif choice == 6:
    print("Exiting Program...")
    break

# Handle invalid menu choices
else:
    print("Invalid Choice. Please try again.")

"""Explanation in Comments"""

# A set stores only unique values and automatically removes duplicates.

# add() is used to add an item to the set.

# discard() removes an item if it exists.
# It does not generate an error if the item is not found.

# clear() removes all elements from the set.

# The 'in' operator checks whether an item exists in the set.

# A while loop is used to display the menu repeatedly
# until the user chooses to exit.
```

---